

Registros, Ponteiros e Alocação Dinâmica

Os Blocos de Construção de Estruturas de Dados

Uma Abordagem Comparativa entre C e Python

Disciplina: ALGORITMOS E ESTRUTURAS DE DADOS 1

Professora: Profa Dra Renata Dutra Braga

Semestre: 2025/2

Roteiro da Aula

Nesta aula, vamos compreender os conceitos fundamentais para a criação de estruturas de dados, comparando a gestão de memória manual (C) com a automática (Python).

1

Registros

Agrupando dados relacionados (struct em C vs. class em Python)

2

Ponteiros vs. Referências

Manipulando endereços em C e objetos em Python

3

Alocação Dinâmica

malloc/free em C vs. Instanciação de objetos em Python

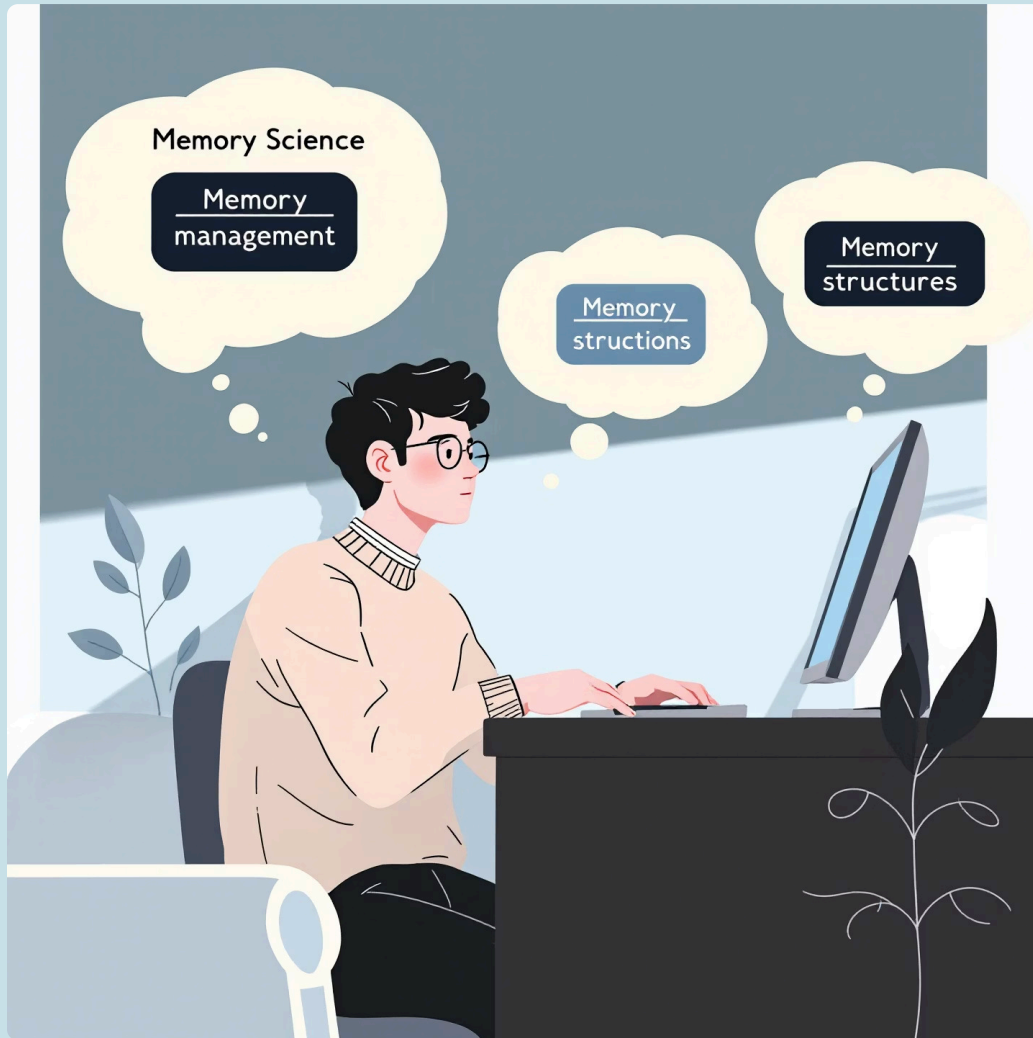
4

Aplicação

Criando a base para Listas Encadeadas em ambas as linguagens

Objetivo: Entender as diferenças fundamentais na forma como C e Python lidam com dados estruturados e gerenciamento de memória.

Por que Estudar Estes Conceitos?



Compreender registros, ponteiros e alocação dinâmica é fundamental porque:

- São a **base para todas as estruturas de dados** que estudaremos no curso
- Revelam como os dados são realmente organizados na memória do computador
- Permitem entender as **diferenças de desempenho** entre diferentes implementações
- Ajudam a explicar por que algumas linguagens são mais rápidas e outras mais fáceis de usar

Você não precisa ser um expert em gerenciamento de memória, mas entender os fundamentos irá fazer toda a diferença em sua formação!

Parte 1: Registros – Agrupando Dados

A necessidade de agrupar diferentes tipos de dados em uma única entidade é universal na programação.

Em C: struct

Um tipo de dado que agrupa variáveis sob um único nome. É um "molde" para um bloco de memória.

```
// Definindo o 'molde'
struct Aluno {
    char nome[80];
    int matricula;
    float ira;
};

// Criando uma variável (alocação estática)
struct Aluno aluno1;
aluno1.matricula = 20251234;
```

Em Python: class

Python usa classes para este fim. Uma classe é um "molde" para criar objetos, que agrupam dados (atributos) e comportamentos (métodos).

```
# Definindo a classe (molde)
class Aluno:
    def __init__(self, nome, matricula, ira):
        self.nome = nome
        self.matricula = matricula
        self.ira = ira

# Criando um objeto (instância da classe)
aluno1 = Aluno("Maria Souza", 20251234, 9.2)
print(aluno1.matricula)
```

Enquanto em C temos um foco maior apenas nos dados, em Python já temos também os comportamentos (métodos) incorporados.

A Ideia Central dos Registros

Imagine que você precisa armazenar informações sobre vários alunos. Seria muito complicado e propenso a erros criar variáveis separadas para cada atributo de cada aluno:

```
// Abordagem problemática  
char nome_aluno1[80], nome_aluno2[80];  
int matricula_aluno1, matricula_aluno2;  
float ira_aluno1, ira_aluno2;
```

A solução é **agrupar os dados relacionados**, seja com `struct` em C ou `class` em Python, para:

- Organizar logicamente os dados relacionados
- Facilitar a passagem desses dados para funções
- Tornar o código mais legível e manutenível
- Permitir a criação de múltiplas "instâncias" com a mesma estrutura



Parte 2: Ponteiros (C) vs. Referências (Python)

A ideia é ter uma variável que "aponte" para outra informação. A forma como isso é exposto ao programador varia drasticamente.

Em C: Ponteiros Explícitos

- O programador manipula diretamente os endereços de memória
- Usa os operadores & (endereço de) e * (conteúdo de)

```
int nota = 10;
int *ptr_nota; // ponteiro para um inteiro
ptr_nota = &nota; // recebe o ENDEREÇO

printf("Valor: %d\n", *ptr_nota); // Acessa o valor
```

Em Python: Referências Implícitas

- Não existem "ponteiros" da mesma forma
- Toda variável é um nome que se refere a um objeto

```
# 'notas' é uma lista (objeto) na memória
notas = [10, 9, 8]

# 'minhas_notas' não cria uma nova lista!
# Apenas se refere ao MESMO objeto
minhas_notas = notas

minhas_notas[0] = 5
print(notas) # Saída: [5, 9, 8]
```



Visualizando Ponteiros e Referências

Em C

Um ponteiro armazena literalmente o **endereço de memória** (como 0x7fff5fbff8c) onde o dado está armazenado.

Quando você usa `*ptr`, está dizendo: "vá até este endereço e me traga o valor que está lá".

Em Python

As variáveis são como **etiquetas** ou **rótulos** colados nos objetos, não "caixas" que contêm valores.

Quando você faz `b = a`, está apenas criando um novo rótulo para o mesmo objeto, não copiando o objeto.

Esta diferença fundamental explica por que em Python, mudar um objeto através de uma variável afeta todas as outras variáveis que se referem ao mesmo objeto.

Parte 3: Alocação Dinâmica de Memória

Criar espaço para dados em tempo de execução é essencial quando não sabemos de antemão o tamanho ou quantidade de dados que vamos manipular.

Em C: Manual (malloc e free)

```
// Aloca memória para um 'struct Aluno'
struct Aluno *aluno_novo;
aluno_novo = (struct Aluno*) malloc(sizeof(struct Aluno));

// Usa o ponteiro para acessar os campos
aluno_novo->matricula = 20259876;

// ...

// OBRIGATÓRIO: Libera a memória
free(aluno_novo);
```

Em Python: Automática (Garbage Collector)

```
# Aloca memória para um objeto Aluno
aluno_novo = Aluno("Carlos Pereira", 20259876, 8.8)

# Usa o objeto normalmente
print(aluno_novo.matricula)

# Para "liberar", basta remover as referências
# O Garbage Collector fará o resto
aluno_novo = None
```

A alocação dinâmica permite que nossos programas usem apenas a memória que realmente precisam, crescendo e encolhendo conforme necessário durante a execução.

Os Perigos da Alocação Manual em C

Memory Leak (Vazamento de Memória)

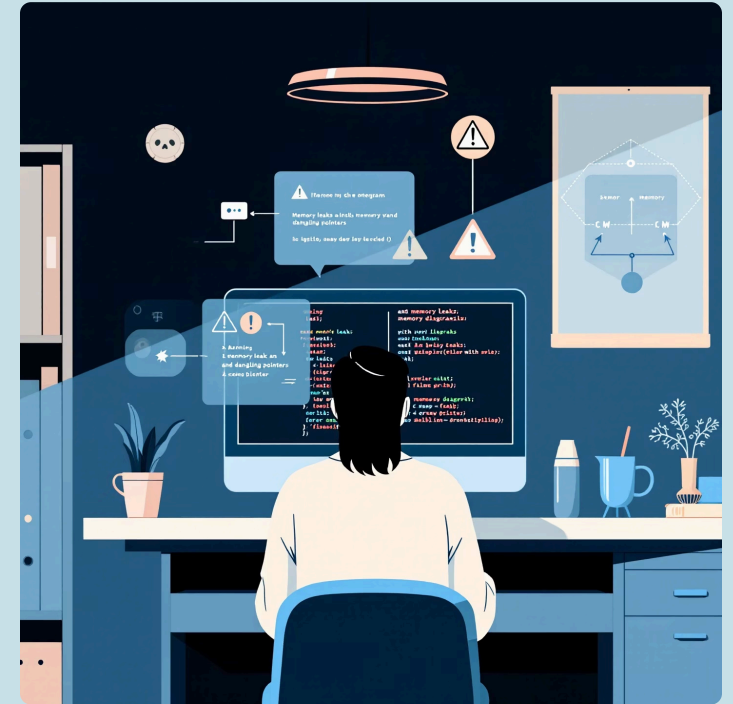
Quando você aloca memória com `malloc()` mas esquece de liberá-la com `free()`, a memória fica inacessível mas ainda reservada.

```
void funcao() {  
    int *p = malloc(100 * sizeof(int));  
    // Esqueceu do free()!  
} // p sai de escopo, mas a memória continua alocada
```

Dangling Pointer (Ponteiro Pendurado)

Quando você libera memória com `free()` mas depois tenta acessá-la através do mesmo ponteiro.

```
int *p = malloc(sizeof(int));  
*p = 10;  
free(p); // Libera a memória  
*p = 20; // ERRO! Acesso a memória já liberada
```



Estes erros são comuns e difíceis de detectar, pois podem não causar problemas imediatos.

Python evita estes problemas com seu **gerenciamento automático de memória**, mas com um pequeno custo de desempenho.

Parte 4: Aplicação – Criando um "Nó" para Lista Encadeada

Vamos aplicar os conceitos aprendidos para definir a estrutura básica que permitirá construir listas encadeadas: um nó que contém um dado e uma "seta" para o próximo elemento.

Em C: struct auto-referenciada

```
#include

// Um nó que guarda um número e aponta para o próximo
struct No {
    int dado;
    struct No *proximo; // Ponteiro para outra struct do MESMO
TIPO
};

// Criando e conectando dois nós dinamicamente
struct No *no1 = (struct No*) malloc(sizeof(struct No));
struct No *no2 = (struct No*) malloc(sizeof(struct No));

no1->dado = 10;
no1->proximo = no2; // no1 "aponta" para no2

no2->dado = 20;
no2->proximo = NULL; // NULL indica o fim da lista

// ... usar free() em ambos no final
free(no1);
free(no2);
```

Em Python: class com auto-referência

```
# Um nó que guarda um dado e referencia o próximo
class No:
    def __init__(self, dado):
        self.dado = dado
        self.proximo = None # None é o análogo de NULL

# Criando e conectando dois nós (objetos)
no1 = No(10)
no2 = No(20)

no1.proximo = no2 # 'proximo' de no1 referencia o objeto no2

# A memória será liberada automaticamente
# quando no1 e no2 saírem de escopo
```

Visualizando um Nó e uma Lista Encadeada Simples



A estrutura de nó é o bloco fundamental para construir muitas estruturas de dados complexas, como:

Listas Encadeadas

- Simples
- Duplas
- Circulares

Árvores

- Binárias
- AVL
- B-Trees

Grafos

- Listas de adjacência
- Matriz de adjacência

Estas estruturas são implementadas de maneira mais direta em C, mas os mesmos princípios se aplicam quando trabalhamos com Python, apenas com uma sintaxe mais limpa e segura.

Resumo Comparativo: C vs. Python

Conceito	C	Python
Agrupar Dados	struct	class
"Apontador"	Ponteiro explícito (*ptr)	Variável como referência/rótulo
Acesso via "Apontador"	Operador seta (->)	Operador ponto (.)
Alocação de Memória	Manual com malloc()	Automática na criação do objeto
Liberação de Memória	Manual com free()	Automática via Garbage Collector

C oferece **controle de baixo nível** sobre a memória, resultando em desempenho otimizado, mas aumenta a complexidade e o risco de erros. Python **abstrai o gerenciamento de memória**, tornando o desenvolvimento mais rápido e seguro, em troca de um pequeno overhead de desempenho.

Diferenças: C vs. Python

1 C

- Controle explícito da memória
- Sintaxe mais verbosa
- Execução mais rápida
- Maior possibilidade de erros
- Compilada para código nativo

2 Ambos

- Podem implementar as mesmas estruturas de dados
- Usam ponteiros/referências para evitar cópias
- Permitem definir tipos personalizados
- Suportam alocação dinâmica

3 Python

- Garbage collector automático
- Sintaxe mais limpa e legível
- Execução mais lenta
- Menor risco de erros de memória
- Interpretada (bytecode)

Embora os conceitos sejam semelhantes, as filosofias das linguagens são diferentes: C prioriza o **desempenho e controle**, enquanto Python prioriza a **produtividade do programador e segurança**.

Por Que Aprender os Dois Modelos?



Compreensão

Entender como a memória é gerenciada em C nos dá ideias sobre o que acontece "por baixo dos panos" em todas as linguagens.



Versatilidade

Cada modelo tem seus pontos fortes. Saber quando usar cada um é uma habilidade valiosa para um cientista da computação.



Visão Completa

Ao comparar as abordagens, você desenvolve uma compreensão mais completa dos prós e contras de cada paradigma de programação.



A comparação entre C e Python ilustra a evolução das linguagens de programação e os trade-offs entre desempenho e facilidade de uso.

Na disciplina de Algoritmos e Estruturas de Dados, essa dupla perspectiva nos permite [entender tanto o funcionamento interno quanto a aplicação prática](#) das estruturas que estudaremos.

Próximos Passos



Nas próximas aulas:

- Correção da lista de exercícios
- Tipos Abstratos de Dados (TADs)/

Exercícios práticos:

Exercício 1: Defina uma estrutura de dados Retangulo capaz de armazenar dois valores de ponto flutuante: largura e altura. Em seu programa principal, crie uma instância dessa estrutura, atribua valores para suas dimensões e, em seguida, calcule e imprima o valor de sua área.

Exercício 2: Crie uma função chamada redimensionar que receba como parâmetros uma referência/ponteiro para uma estrutura Retangulo (criada no exercício anterior) e um fator de escala numérico. A função deve alterar as dimensões do retângulo original, multiplicando sua largura e altura pelo fator recebido. No programa principal, demonstre que a função modifica o retângulo original imprimindo sua área antes e depois da chamada da função.

Exercício 3: Defina uma estrutura de dados Ponto para armazenar coordenadas x e y inteiras. No programa principal, aloque dinamicamente na memória o espaço para uma única instância de Ponto. Atribua valores às suas coordenadas, imprima-as na tela e, ao final, garanta que a memória alocada seja devidamente liberada.

Leituras recomendadas: Capítulos 1 e 2 do livro "Estruturas de Dados e Algoritmos em C" de Thomas Cormen.